

Network Security Lab: Attack Simulation and IDS/IPS Evaluation

Alexander Schoolcraft

Richard Zheng

April 27, 2026

1 Introduction

Modern network environments face a continuously evolving threat landscape, ranging from automated reconnaissance and brute-force authentication attempts to exploitation of vulnerable services and lateral movement within internal networks. Traditional security mechanisms, such as static firewalls and rule-based filtering, are often insufficient to detect and respond to these increasingly sophisticated and dynamic attack patterns. As a result, Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) have become critical components of modern network defense strategies, providing visibility into network traffic and enabling active mitigation of malicious activity.

An IDS operates by monitoring network traffic and generating alerts based on known signatures or anomalous behavior, allowing administrators to identify potential threats. However, IDS solutions alone are limited in that they do not actively prevent attacks. In contrast, an IPS extends this capability by operating inline with network traffic, enabling real-time inspection and enforcement actions such as packet dropping or connection termination. The transition from detection to prevention introduces important trade-offs, including potential impacts on network performance, false positives, and system complexity.

This project presents the design and implementation of a controlled virtual network environment to simulate common network-based attacks and evaluate the effectiveness of IDS and IPS mechanisms. The environment consists of an external attacker system, an internal network containing vulnerable and standard hosts, and a gateway device configured with Suricata to function as both an IDS and an IPS. The network is intentionally segmented to emulate a simplified enterprise architecture, with all traffic between external and internal systems routed through the inspection point.

Within this environment, representative attack scenarios are executed, including network reconnaissance via port scanning and credential-based attacks through brute-force authentication attempts. These attacks are selected due to their prevalence in real-world threat activity and their suitability for controlled experimentation. Each attack is evaluated under two conditions: (1) IDS/logging-only operation, and (2) IPS-enabled enforcement. This structured approach enables direct comparison of detection capability, attack success rate, and the impact of preventive controls.

The primary objective of this work is to analyze how effectively an inline IDS/IPS system can detect and mitigate common attack techniques while maintaining normal network functionality. By capturing and analyzing traffic, alert data, and system behavior across multiple configurations, this study provides insight into the practical strengths and limitations of deploying Suricata as a network security control. Additionally, the project emphasizes reproducibility and clarity, ensuring that the experimental setup and results can be independently validated and extended. Importantly, this work does not attempt to develop novel detection algorithms, but rather evaluates practical deployment considerations, effectiveness of existing frameworks, and strengths and weaknesses of an IDS vs. an IPS.

The remainder of this paper is organized as follows: Section 2 describes the network design and architecture of the lab environment, including topology and addressing schemes. Section 3 details the tools and environment configuration used to implement the system. Subsequent sections present the attack scenarios, defensive mechanisms, experimental methodology, re-

sults, and analysis.

2 Network Design & Architecture

The experimental environment was designed to emulate a simplified enterprise network architecture consisting of an external network (WAN), an internal local area network (LAN), and a centralized inspection point responsible for traffic monitoring and control. The primary objective of this design is to ensure that all communication between external and internal systems is routed through a single gateway device, enabling comprehensive visibility and enforcement of security policies.

2.1 Topology Overview

The network topology, illustrated in Figure 1, consists of four primary systems: an external attacker host, an external device to act as legitimate traffic, an intrusion detection and prevention system (IDPS) gateway, and an internal client host to act as the target. The external systems simulate untrusted network sources, while the internal network contains a vulnerable host.

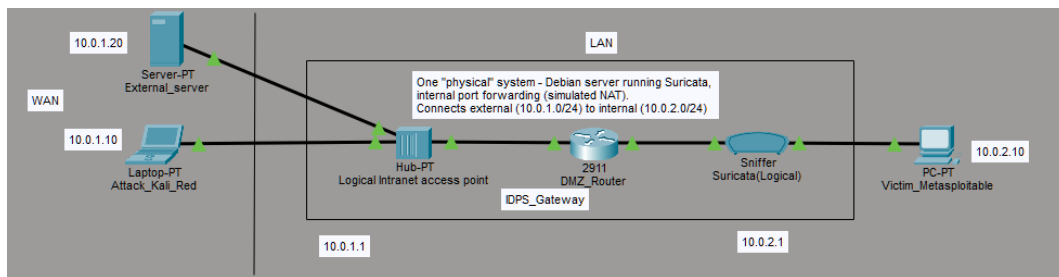


Figure 1: Simulated Network Topology

All traffic between these segments is routed through the IDPS_Gateway, ensuring that no direct communication occurs between external and internal systems without inspection.

2.2 System Roles

Each system in the environment serves a distinct role within the network:

- **Attack_Kali_Red (Kali Linux):** Simulates an external adversary and is used to perform reconnaissance and active attacks, including port scanning and brute-force authentication attempts.
- **External_Server (Debian Server):** Simulates benign external network traffic representative of typical internet activity, to produce standard legitimate traffic to ensure that non-malicious communication was not hindered when the IPS was in place.
- **IDPS_Gateway (Ubuntu Server with Suricata):** Acts as the central routing and inspection point. This system performs packet forwarding between network segments while simultaneously monitoring traffic using Suricata. It operates in both IDS and IPS modes during different phases of the experiment.
- **Victim_Metasploitable (Metasploitable):** Represents a deliberately insecure system containing known vulnerabilities and weak authentication mechanisms. It serves as the primary target for attack scenarios.

2.3 Network Segmentation and Routing

The network is segmented into two isolated subnets to enforce controlled communication paths:

- External Network: 10.0.1.0/24
- Internal Network: 10.0.2.0/24

The IDPS_Gateway is configured with two network interfaces, one connected to each subnet. This dual-homed configuration enables the system to function as a router, forwarding packets between networks while also serving as the inspection point for all traffic.

IP forwarding is enabled on the gateway system, ensuring that packets can traverse between the external and internal networks. Static routing is configured on the attacker and internal systems to direct traffic through the gateway, preventing any bypass of the inspection mechanism.

2.4 Addressing Scheme

The IP addressing scheme used in the environment is summarized in Table 1.

System	IP Address	Role
Attack_Kali_Red	10.0.1.10	External attacker
External_Server	10.0.1.20	External "internet" simulator
IDPS_Gateway (External Interface)	10.0.1.1	Gateway (WAN side)
IDPS_Gateway (Internal Interface)	10.0.2.1	Gateway (LAN side)
Victim_Metasploitable	10.0.2.10	Vulnerable host

Table 1: Network Addressing Scheme

2.5 Design Rationale

The chosen architecture emphasizes simplicity, control, and reproducibility while maintaining sufficient realism to reflect common enterprise network structures. By enforcing a single inspection point, the design ensures that all relevant traffic can be monitored and analyzed without requiring complex switching or mirroring configurations.

The use of isolated internal networks within a virtualized environment eliminates external noise and uncontrolled variables, allowing for precise measurement of attack and defense behavior. Additionally, the separation of vulnerable and standard internal hosts enables comparative analysis of how different systems are affected by both attacks and defensive mechanisms.

Finally, the decision to implement the IDPS as an inline gateway rather than a passive

monitoring system allows for direct evaluation of both detection (IDS) and prevention (IPS) capabilities within the same environment. This design supports a structured experimental approach and facilitates clear comparison between different operational modes.

3 Tools & Environment Setup

This section describes the tools, systems, and configurations used to construct the experimental environment. The design prioritizes reproducibility, controlled conditions, and clear separation of roles across systems.

3.1 Virtualization Platform

The entire environment was implemented using Oracle VirtualBox, allowing multiple isolated systems to be deployed on a single host machine. Virtualization enables controlled network segmentation, repeatable experiments, and safe execution of potentially harmful activities such as exploitation and brute-force attacks.

Two internal virtual networks were created within VirtualBox to simulate external and internal network segments:

- **NetSecExt:** Represents the external (WAN) network
- **NetSecInt:** Represents the internal (LAN) network

All virtual machines were connected exclusively to these internal networks, ensuring complete isolation from the host system and external internet traffic during experimentation.

3.2 Network Configuration

The network was configured using static IP addressing and manual routing to ensure deterministic behavior. The IDPS_Gateway system was configured with two network interfaces:

- External interface connected to NetSecExt (10.0.1.1/24)
- Internal interface connected to NetSecInt (10.0.2.1/24)

IP forwarding was enabled on the IDPS_Gateway system to allow traffic to pass between the two networks. All systems were configured to route traffic through this gateway, ensuring that all inter-network communication could be monitored and controlled.

To prevent unintended traffic from interfering with experimental results, no network address translation (NAT) or external connectivity was maintained during testing phases. All artifacts generated within the lab were retrieved through a shared folder on the host and IDPS_Gateway.

3.3 Security Tools

The primary security tool used in this project is Suricata, an open-source network intrusion detection and prevention system. Suricata was deployed on the IDPS_Gateway system and configured to operate in two modes:

- **Intrusion Detection System (IDS):** In this mode, Suricata passively monitors network traffic and generates alerts based on known signatures and behavioral patterns.
- **Intrusion Prevention System (IPS):** In this mode, Suricata operates inline with network traffic and is capable of actively blocking malicious packets.

Suricata rules were obtained and updated using the `suricata-update` utility, ensuring that the detection engine had access to current threat signatures.

Additionally, `iptables` was used on the IDPS_Gateway system to facilitate traffic handling and to enable integration with Suricata in IPS mode through packet queuing mechanisms.

3.4 Attack Tools

The Attack_Kali_Red system was used to execute all attack scenarios. The following tools were utilized:

- **Nmap:** Used for network reconnaissance and port scanning to identify open services and potential targets.
- **Hydra:** Used for brute-force authentication attacks against services such as SSH.
- **Command-line tools:** Post-compromise, SSH and SCP were used to locate and extract "sensitive" data.

These tools were selected due to their widespread use in both penetration testing and real-world attack scenarios, as well as their compatibility with signature-based detection systems such as Suricata.

3.5 System Preparation and Configuration

Each system was configured to ensure stable and repeatable operation. Static IP addresses were assigned using system-specific network configuration tools, and unnecessary background services were minimized to reduce noise in network traffic.

Logging mechanisms were enabled on all systems to support analysis, including system logs on the target machines and alert logs generated by Suricata. Time synchronization was configured to ensure accurate correlation between attack activity and detection events.

Snapshots of each virtual machine were created after initial configuration to allow rapid restoration of the environment to a known state between experimental runs.

4 Attack Implementation

4.1 Assumptions and Pre-Attack Conditions

In real-world attack scenarios, adversaries rarely begin with complete knowledge of valid user credentials. Instead, early stages of an attack often involve the identification or inference of valid usernames prior to attempting authentication attacks. Common techniques for obtaining usernames include open-source intelligence (OSINT), social engineering, enumeration of public-facing services, and inference based on organizational naming conventions (e.g., first initial and last name formats).

For the purposes of this study, the username used in the brute-force authentication phase was assumed to be known. This abstraction was intentionally introduced to reduce experimental complexity and to focus analysis on the effectiveness of intrusion detection and prevention mechanisms against credential-based attacks.

This assumption reflects a realistic condition in which an attacker has already obtained partial information, such as a valid username, through prior reconnaissance or compromise. By isolating the authentication and post-compromise phases, the experiment ensures that observed results are directly attributable to detection and prevention capabilities rather than variability in user enumeration techniques.

While username discovery is an important component of the overall attack lifecycle, it is outside the scope of this work. Future extensions of this environment could incorporate user enumeration techniques to provide a more comprehensive end-to-end attack simulation.

For this demonstration, the attacker is assumed to have external network access, the ability to perform active reconnaissance, and the capability to execute automated credential-based attacks, but does not initially possess internal network access or privileged credentials.

4.2 Target Enumeration

The initial phase of the attack involved network reconnaissance using Nmap to identify active hosts and exposed services within the internal network. A TCP SYN scan was conducted against the target network using the command `nmap -sS 10.0.2.0/24` to find potential targets.

As shown in Appendix A, the scan successfully identified multiple open ports on the victim system, including SSH (22), HTTP (80), and MySQL (3306), indicating the presence of accessible services and potential attack surfaces.

4.3 Attack Execution

Following enumeration, a brute-force authentication attack was conducted against the SSH service using Hydra. A reduced wordlist was used to ensure controlled and repeatable testing conditions. Specifically, the top 100 entries from the `10k-most-common.txt` wordlist (from the SecLists repository) were selected, with the known valid password (“pass”) intentionally placed at the end of the list to simulate a realistic worst-case scenario for detection and prevention mechanisms.

The attack was executed using the command `hydra -l victim -P testlist.txt -t 4 ssh://10.0.2.10`. As shown in Appendix B, the attack successfully recovered valid credentials for the target system.

Using the compromised credentials, the attacker established an SSH session with the Victim.Metasploitable system. Post-compromise activities included local reconnaissance, identification of a hidden file containing simulated sensitive data, and subsequent exfiltration of that file using SCP.

This sequence of actions demonstrates a complete attack chain, progressing from reconnaissance to credential compromise and ultimately to data exfiltration. The full command sequence and outputs are provided in Appendix C.

5 Defense Mechanisms

5.1 Suricata Configuration and IPS Integration

Following the initial attack phase, the IDPS_Gateway system was reconfigured to operate as an inline intrusion prevention system (IPS) using Suricata in conjunction with the Linux netfilter framework.

This configuration involved enabling NFQUEUE support within the Suricata configuration file (`/etc/suricata/suricata.yaml`), allowing the system to process packets forwarded from the kernel and issue accept or drop decisions in real time.

Traffic was then redirected through Suricata using a series of `iptables` rules designed to both enforce security policies and facilitate packet inspection. The applied rules are shown below:

- Reset existing forwarding rules:

```
– sudo iptables -F
– sudo iptables -P FORWARD ACCEPT
```

- Configure SSH brute-force detection and blocking:

```
– sudo iptables -I FORWARD 1 -p tcp --dport 22 -m conntrack --ctstate NEW
–m recent --name SSHBRUTE --set
– sudo iptables -I FORWARD 2 -p tcp --dport 22 -m conntrack --ctstate NEW
–m recent --name SSHBRUTE --update --seconds 10 --hitcount 4 -j DROP
```

- Configure port scan detection and blocking:

```
– sudo iptables -I FORWARD 3 -p tcp -m conntrack --ctstate NEW -m recent
–name PORTSCAN --set
– sudo iptables -I FORWARD 4 -p tcp -m conntrack --ctstate NEW -m recent
–name PORTSCAN --update --seconds 5 --hitcount 20 -j DROP
```

- Redirect remaining traffic to Suricata for inspection:

```
– sudo iptables -A FORWARD -j NFQUEUE --queue-num 0 --queue-bypass
```

These rules implement rate-based detection of malicious behavior, allowing the system to identify and block high-frequency connection attempts characteristic of brute-force authentication attacks and network scanning activity. Legitimate traffic, which occurs at significantly lower rates, is allowed to pass without interruption.

Finally, Suricata was launched in inline mode and attached to the NFQUEUE using the command:

- `sudo suricata -c /etc/suricata/suricata.yaml -q 0`

This configuration enabled a layered defense model in which Suricata provides detection and analysis capabilities, while `iptables` ensures consistent enforcement of blocking actions.

5.2 Attack Execution with IPS Enabled

Once the IPS configuration was verified, the attack sequence described in the previous section was repeated under identical conditions.

In contrast to the IDS-only scenario, the attack was unsuccessful. Network reconnaissance using `nmap` was unable to identify open ports on the target system, with all scanned ports appearing as filtered. Similarly, the brute-force authentication attempt using `hydra` failed to recover valid credentials due to repeated connection attempts being blocked by the gateway.

As a result, no unauthorized access to the victim system was achieved, and no data exfiltration could be performed. This demonstrates that the IPS configuration effectively disrupted both the reconnaissance and exploitation phases of the attack.

Detailed logs corresponding to this phase of the experiment are provided in Appendices D and E.

Additionally, during this phase, non-malicious traffic was generated using `curl` between Victim.Metasploitable and External.Server to validate that legitimate communication was not disrupted by the IPS configuration.

As shown in Figure 2, normal HTTP communication was successfully established, including a TCP SYN, SYN/ACK, and ACK exchange followed by a valid HTTP 200 OK response. This confirms that the implemented IPS selectively blocked malicious activity while allowing benign traffic to pass without interruption.

6134	0.000330	10.0.2.10	10.0.2.10	TCP	74	41590	80	41590 → 80 [SYN] Seq=64240 Len=0 MSS=1460 SACK_PERM TSval=680402422 TSecr=0 WS=128
6135	0.000232	PCSSystemtec_3frc3i...	Broadcast	ARP	42			who has 10.0.1.20? Tell 10.0.1.1
6136	0.000555	PCSSystemtec_3a19b...	PCSSystemtec_3frc3i...	ARP	60			10.0.1.20 is at 08:00:27:3a:9b:dc
6137	0.000000	10.0.2.10	10.0.1.20	TCP	74	80	41590	80 → 41590 [SYN, ACK] Seq=64240 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=4513079 TSecr=680402422 WS=128
6138	0.000495	10.0.1.20	10.0.2.10	TCP	66	41590	80	41590 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=680402426 TSecr=4513079
6139	0.000704	10.0.1.20	10.0.2.10	HTTP	139	41590	80	GET / HTTP/1.1
6140	0.001059	10.0.2.10	10.0.1.20	TCP	66	80	41590	80 → 41590 [ACK] Seq=1 Ack=74 Win=29056 Len=0 TSval=4513080 TSecr=680402427
6141	0.001140	10.0.2.10	10.0.1.20	TCP	1514	80	41590	80 → 41590 [ACK] Seq=1 Ack=74 Win=29056 Len=1448 TSval=4513080 TSecr=680402427 [TCP PDU reassembled in 6142]
6142	0.000261	10.0.2.10	10.0.1.20	HTTP	136	80	41590	HTTP/1.1 200 OK (text/html)
6143	0.000311	10.0.1.20	10.0.2.10	TCP	66	41590	80	41590 → 80 [ACK] Seq=74 Ack=1519 Win=62848 Len=0 TSval=680402430 TSecr=4513080
6144	0.000401	10.0.1.20	10.0.2.10	TCP	66	41590	80	41590 → 80 [FIN, ACK] Seq=74 Ack=1519 Win=62848 Len=0 TSval=680402430 TSecr=4513080
6145	0.001221	10.0.2.10	10.0.1.20	TCP	66	80	41590	80 → 41590 [FIN, ACK] Seq=1519 Ack=75 Win=29056 Len=0 TSval=4513080 TSecr=680402430
6146	0.000097	10.0.1.20	10.0.2.10	TCP	66	41590	80	41590 → 80 [ACK] Seq=75 Ack=1520 Win=62848 Len=0 TSval=680402433 TSecr=4513080
6147	5.216930	PCSSystemtec_3a19b...	PCSSystemtec_3frc3i...	ARP	60			who has 10.0.1.1? Tell 10.0.1.20
6148	0.000010	PCSSystemtec_3frc3i...	PCSSystemtec_3a19b...	ARP	42			10.0.1.1 is at 08:00:27:3f:c3:70
6149	11.232092	10.0.2.10	10.0.1.20	TCP	74	51621	80	51621 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM TSval=4517193 TSecr=0 WS=128
6150	0.000739	10.0.1.20	10.0.2.10	TCP	74	80	51621	80 → 51621 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=680418804 TSecr=4517193 WS=128
6151	0.001754	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=4517194 TSecr=680418804
6152	0.001414	10.0.2.10	10.0.1.20	HTTP	139	51621	80	GET / HTTP/1.1
6153	0.000480	10.0.1.20	10.0.2.10	TCP	66	80	51621	80 → 51621 [ACK] Seq=1 Ack=74 Win=65152 Len=0 TSval=680418808 TSecr=4517194
6154	0.000279	10.0.1.20	10.0.2.10	TCP	7306	80	51621	80 → 51621 [PSH, ACK] Seq=1 Ack=74 Win=65152 Len=7240 TSval=680418891 TSecr=4517194 [TCP PDU reassembled in 6155]
6155	0.000000	10.0.1.20	10.0.2.10	HTTP	3784	80	51621	HTTP/1.1 200 OK (text/html)
6156	0.004140	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=2097 Win=35072 Len=0 TSval=4517196 TSecr=680418891
6157	0.000063	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=4345 Win=37888 Len=0 TSval=4517196 TSecr=680418891
6158	0.000057	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=5793 Win=40832 Len=0 TSval=4517196 TSecr=680418891
6159	0.000055	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=7241 Win=3776 Len=0 TSval=4517196 TSecr=680418891
6160	0.000053	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=8689 Win=46592 Len=0 TSval=4517196 TSecr=680418891
6161	0.000049	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=18137 Win=49536 Len=0 TSval=4517196 TSecr=680418891
6162	0.001236	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=18959 Win=52480 Len=0 TSval=4517197 TSecr=680418891
6163	0.038001	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [FIN, ACK] Seq=74 Ack=18959 Win=52480 Len=0 TSval=4517205 TSecr=680418891
6164	0.000042	10.0.1.20	10.0.2.10	TCP	66	80	51621	80 → 51621 [FIN, ACK] Seq=18959 Ack=75 Win=65152 Len=0 TSval=680418936 TSecr=4517205
6165	0.001501	10.0.2.10	10.0.1.20	TCP	66	51621	80	51621 → 80 [ACK] Seq=74 Ack=18960 Win=52480 Len=0 TSval=4517207 TSecr=680418936

Figure 2: Packet capture demonstrating successful HTTP communication between External.Server and Victim.Metasploitable, including TCP handshake, HTTP request, and valid response under IPS enforcement

6 Experimental Setup

To evaluate the effectiveness of IDS and IPS configurations, a structured experimental approach was used. Each attack scenario was executed under two distinct conditions:

- **IDS Mode:** Suricata configured for detection-only, allowing all traffic to pass while generating alerts.
- **IPS Mode:** Suricata configured inline with iptables enforcement, enabling active blocking of malicious traffic.

Each experiment followed the same sequence of actions:

1. Network reconnaissance using Nmap

2. Brute-force authentication using Hydra
3. Post-compromise activity including file discovery and exfiltration

Traffic was captured at the IDPS Gateway using packet capture tools, and Suricata logs were recorded for analysis. The environment was reset between runs to ensure consistency and eliminate residual state from previous experiments.

Additionally, controlled legitimate traffic was generated using HTTP requests to validate that defensive mechanisms did not disrupt normal communication.

7 Results & Analysis

7.1 IDS Mode Results

In IDS mode, Suricata successfully detected both reconnaissance and brute-force activity. Alerts corresponding to port scanning and repeated SSH authentication attempts were observed in the logs.

However, these detections did not prevent the attack from succeeding. The Nmap scan successfully identified open ports on the target system, and the Hydra attack recovered valid credentials. This allowed the attacker to gain unauthorized access and perform data exfiltration.

7.2 IPS Mode Results

In IPS mode, the system demonstrated effective disruption of both reconnaissance and attack phases.

Nmap scanning results showed all ports on the target system as filtered, indicating that scanning activity was successfully blocked. Similarly, the Hydra brute-force attack failed to recover valid credentials, as repeated connection attempts were dropped by the gateway.

Suricata logs confirmed detection of malicious activity, while iptables enforcement ensured consistent blocking behavior.

7.3 Legitimate Traffic Validation

To verify that defensive mechanisms did not interfere with normal operations, HTTP traffic was generated between the external server and the internal system.

Packet capture analysis (Figure 2) demonstrates successful TCP handshakes and valid HTTP responses, confirming that legitimate communication was preserved while malicious activity was blocked.

While quantitative metrics such as false positive rate and throughput impact were not formally measured, qualitative analysis of traffic and logs indicates high detection accuracy with minimal disruption to legitimate traffic. Future work could incorporate detailed performance benchmarking.

8 Discussion

The results highlight key differences between intrusion detection and intrusion prevention systems. While IDS configurations provide valuable visibility into network activity, they do not inherently prevent attacks from succeeding.

In contrast, IPS configurations introduce enforcement capabilities that can actively disrupt malicious behavior. However, this effectiveness depends on proper tuning of detection thresholds and enforcement mechanisms.

Early testing revealed that threshold-based detection alone may allow attacks to succeed if critical actions occur before enforcement is triggered. This limitation emphasizes the importance of integrating detection systems with reliable enforcement mechanisms such as firewall rules.

Additionally, encrypted protocols such as SSH limit visibility into payload data, requiring

reliance on behavioral indicators rather than content inspection.

The layered approach implemented in this study—combining Suricata with iptables—proved effective in achieving both detection and prevention while maintaining normal network functionality. This reflects real-world network security architectures, where multiple controls operate together to provide defense in depth.

9 Conclusion & Future Work

This study demonstrated the design and implementation of a controlled network environment for evaluating IDS and IPS effectiveness using Suricata.

Results showed that while IDS configurations successfully detect malicious activity, they do not prevent compromise. In contrast, IPS configurations effectively blocked reconnaissance and brute-force attacks, preventing unauthorized access and data exfiltration.

Importantly, the IPS implementation preserved legitimate network functionality, demonstrating selective enforcement of security controls.

Future work may include expanding the attack scenarios to include additional techniques such as lateral movement, exploitation of application vulnerabilities, and user enumeration. Additionally, further refinement of detection rules and evaluation of performance impacts under higher traffic loads would provide deeper insight into real-world deployment considerations.

A Target Enumeration/NMAP output

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-25 12:39 -0400
Nmap scan report for 10.0.2.1
Host is up (0.00051s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
```

```
22/tcp open  ssh
```

```
Nmap scan report for 10.0.2.10
```

```
Host is up (0.0015s latency).
```

```
Not shown: 991 filtered tcp ports (no-response)
```

```
PORT      STATE SERVICE
```

```
21/tcp open  ftp
```

```
22/tcp open  ssh
```

```
80/tcp open  http
```

```
445/tcp open  microsoft-ds
```

```
631/tcp open  ipp
```

```
3000/tcp closed ppp
```

```
3306/tcp open  mysql
```

```
8080/tcp open  http-proxy
```

```
8181/tcp closed intermapper
```

```
Nmap done: 256 IP addresses (2 hosts up) scanned in 14.25 seconds
```

B Brute Force/Hydra output

```
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do
not use in military or secret service organizations, or for
illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).
```

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at
2026-04-24 17:11:10
```

```
[DATA] max 4 tasks per 1 server, overall 4 tasks, 100 login tries (1
:1/p:100), ~25 tries per task
```

```
[DATA] attacking ssh://10.0.2.10:22/
[22][ssh] host: 10.0.2.10    misc: (null)    login: victim    password:
      pass
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at
      2026-04-24 17:11:34
```

C Exfiltration/Bash Script

```
Script started on 2026-04-25 08:40:15-04:00 [TERM="xterm-256color"
      TTY="/dev/pts/1" COLUMNS="235" LINES="52"]

attacker@kali:~$ ssh victim@10.0.2.10
** WARNING: connection is not using a post-quantum key exchange
      algorithm.
** This session may be vulnerable to "store now, decrypt later"
      attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

victim@10.0.2.10's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86\_64)
      * Documentation:  https://help.ubuntu.com/
Last login: Sat Apr 25 12:25:26 2026
victim@metasploitable3-ub1404:~$ ls
victim@metasploitable3-ub1404:~$ ls -al
total 32
drwxr-xr-x  3 victim victim 4096 Apr 25 12:31 .
```

```

drwxr-xr-x 19 root    root    4096 Apr 23 23:16 ..
-rw-----  1 victim  victim    5 Apr 23 23:39 .bash_history
-rw-r--r--  1 victim  victim   220 Apr 23 23:16 .bash_logout
-rw-r--r--  1 victim  victim  3637 Apr 23 23:16 .bashrc
drwx-----  2 victim  victim  4096 Apr 23 23:39 .cache
-rw-r--r--  1 victim  victim   675 Apr 23 23:16 .profile
-rw-rw-r--  1 victim  victim   185 Apr 25 12:31 .super_secret_info.txt
victim@metasploitable3-ub1404:~$ cat .super_secret_info.txt
This is super secret plans for a whole new product - if this gets
    leaked, we are so completely screwed...I really hope that no one
    can take this!

```

```

flag_{D3f3nc35_4r3_r3411y_1mp0r74n7!}

```

```

victim@metasploitable3-ub1404:~$ exit
logout
Connection to 10.0.2.10 closed.

```

```

attacker@kali:~$ scp victim@10.0.2.10:/home/victim/.
    super_secret_info.txt .
** WARNING: connection is not using a post-quantum key exchange
    algorithm.
** This session may be vulnerable to "store now, decrypt later"
    attacks.
** The server may need to be upgraded. See https://openssh.com/pq.
    html
victim@10.0.2.10's password:
.super_secret_info.txt      0%    0    0.0KB/s   --:--  ETA
.super_secret_info.txt     100% 185    63.6KB/s   00:00

```

```
attacker@kali:~$ exit
```

```
Script done on 2026-04-25 08:41:37-04:00 [COMMAND\_EXIT\_CODE="0"]
```

D Suricata log with IPS active

```
04/25/2026-17:27:08.834766 [wDrop] [**] [1:1000099:1] LOCAL TEST
    DROP ICMP to Victim\_Metasploitable [**] [Classification: (null)]
    [Priority: 3] {ICMP} 10.0.1.10:8 -> 10.0.2.10:0
04/25/2026-17:27:35.644596 [**] [1:2006546:9] ET SCAN LibSSH Based
    Frequent SSH Connections Likely BruteForce Attack [**] [
    Classification: Attempted Administrator Privilege Gain] [Priority
    : 1] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.639224 [wDrop] [**] [1:1000001:1] LOCAL DROP
    SSH brute force (libssh pattern) [**] [Classification: (null)] [
    Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.645300 [wDrop] [**] [1:1000001:1] LOCAL DROP
    SSH brute force (libssh pattern) [**] [Classification: (null)] [
    Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.645300 [wDrop] [**] [1:1000001:1] LOCAL DROP
    SSH brute force (libssh pattern) [**] [Classification: (null)] [
    Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.645300 [**] [1:2006546:9] ET SCAN LibSSH Based
    Frequent SSH Connections Likely BruteForce Attack [**] [
    Classification: Attempted Administrator Privilege Gain] [Priority
    : 1] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.684098 [wDrop] [**] [1:1000001:1] LOCAL DROP
    SSH brute force (libssh pattern) [**] [Classification: (null)] [
```

```

Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.686913 [wDrop] [**] [1:1000001:1] LOCAL DROP
SSH brute force (libssh pattern) [**] [Classification: (null)] [
Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.690952 [wDrop] [**] [1:1000001:1] LOCAL DROP
SSH brute force (libssh pattern) [**] [Classification: (null)] [
Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.691005 [wDrop] [**] [1:1000001:1] LOCAL DROP
SSH brute force (libssh pattern) [**] [Classification: (null)] [
Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.693290 [wDrop] [**] [1:1000001:1] LOCAL DROP
SSH brute force (libssh pattern) [**] [Classification: (null)] [
Priority: 3] {TCP} 10.0.1.10:46232 -> 10.0.2.10:22
04/25/2026-17:27:35.914688 [Drop] [**] [1:1000001:1] LOCAL DROP SSH
brute force (libssh pattern) [**] [Classification: (null)] [
Priority: 3] {TCP} 10.0.1.10:46250 -> 10.0.2.10:22
04/25/2026-17:28:53.373461 [**] [1:2034567:1] ET HUNTING curl User-
Agent to Dotted Quad [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {TCP} 10.0.2.10:51621 -> 10.0.1.20:80
04/25/2026-17:28:53.373461 [**] [1:2034567:1] ET HUNTING curl User-
Agent to Dotted Quad [**] [Classification: Potentially Bad
Traffic] [Priority: 2] {TCP} 10.0.2.10:51621 -> 10.0.1.20:80

```

E NMAP and Hydra logs from attacker while IPS active

```

Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-25 13:26 -0400
Nmap scan report for 10.0.2.1

```

Host is up (0.00080s latency).

Not shown: 999 closed tcp ports (reset)

PORT STATE SERVICE

22/tcp open ssh

Nmap scan report for 10.0.2.10

Host is up (0.0041s latency).

All 1000 scanned ports on 10.0.2.10 are in ignored states.

Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 256 IP addresses (2 hosts up) scanned in 13.06 seconds

Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do
not use in military or secret service organizations, or for
illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (<https://github.com/vanhauser-thc/thc-hydra>) starting at
2026-04-25 13:27:23

[DATA] max 4 tasks per 1 server, overall 4 tasks, 100 login tries (1
:1/p:100), ~25 tries per task

[DATA] attacking ssh://10.0.2.10:22/

0 of 1 target completed, 0 valid password found

Hydra (<https://github.com/vanhauser-thc/thc-hydra>) finished at
2026-04-25 13:27:55

F Additional documents submitted with paper

- **netscan.pcap**: PCAP from IDPS_Gateway collected during initial enumeration pass

- **netscan_suricata.log**: Suricata log from IDPS_Gateway from initial enumeration pass
- **brute_force.pcap**: PCAP from IDPS_Gateway collected during initial brute force attack
- **brute_force_suricata.log**: Suricata log from IDPS_Gateway during initial brute force attack
- **exfil.pcap**: PCAP from IDPS_Gateway collected during exfiltration of "secret" document
- **ids_block.pcap**: PCAP from IDPS_Gateway collected after setting up the IPS portion
- **logical_topology.pkt**: Cisco Packet Tracer file containing the logical configuration of the lab network setup
- **set_ids.sh** & **set_ips.sh**: Scripts used to toggle between IDS (logging only, no packet filtering) and IPS (block malicious traffic while allowing normal traffic through) modes
- **startup.sh**: Script used on IDPS_Gateway when booting to ensure forwarding is enabled, share drive is attached, and launch **tmux** to allow for multiple concurrent tools running
- **live_demonstration.mp4**: Screen capture live demonstration of the techniques used in this paper